

Arm Create: AI Optimization Challenge

One challenge. Three frontiers of AI. All powered by Arm.

Welcome to the Arm AI Optimization Challenge 2026. We're inviting developers to build and submit projects that show how AI can be optimized for Arm-powered platforms across three challenge tracks:

- [Physical AI](#): Optimize AI for real-world systems, including robotics, embedded devices, sensors, simulation, autonomy, and edge environments.
- [Cloud AI](#): Optimize AI for scalable infrastructure, including Arm64 cloud, inference performance, frameworks, agents, and production-ready developer workflows.
- [Mobile AI](#): Optimize AI for on-device constraints, including performance, privacy, latency, battery efficiency, and local AI experiences on Arm-powered phones, tablets, and laptops.

Across all tracks, submissions should show clear optimization work and measurable improvements where possible.

Optimizations we will look for:

- Model size: Reduce size on disk or in memory.
- Model quality: Improve fine-tuning or output quality for a given model size.
- Model speed: Improve tokens/sec, time to first token, or other relevant latency metrics.
- Inference server speed: Improve throughput, latency, tokens/sec, or time to first token.
- Developer experience: Improve tools, workflows, setup, documentation, or usability.
- Arm-specific optimization: Implement optimizations in an existing framework, library, model, or application to run better on Arm.

Developers can use [Arm Performix](#) to get exact benchmarks of their Arm based performance and be able to clearly show their results.

Requirements

This year's challenge has three tracks, so you can choose the path that best matches what you want to build. For more details information for each Track, please visit the [Track Details Tab](#).

Submissions to the Hackathon must meet the following requirements:

- Include a Project built with the required developer tools and meets the above Project Requirements.
- Provide a URL to your code repository for judging and testing. The repository must contain all necessary source code, assets, and instructions required for the project to be functional. The repository must be public and open source by including an open source license file. This license should be detectable and visible at the top of the repository page (in the About section).
 - MIT or Apache 2.0

- Include a text description that should explain the features and functionality of your Project.
- Project Overview: A brief description of the project and its purpose. Also explain what makes it interesting and why it should win.
- Functionality / Output: Explain what the project does and what the final output is (optimized model, migration example, scavenger deliverables, etc.).
- Setup Instructions: Step-by-step instructions on how to build/run/validate on an Arm-powered device or Arm64 environment (as applicable to your track).
- Optional: Include a demonstration video of your Project. The video portion of the Submission:
 - should be less than three (3) minutes. Judges are not required to watch beyond three minutes
 - should include footage that shows the Project functioning on the device for which it was built
 - must be uploaded to and made publicly visible on YouTube, Vimeo, or Youku, and a link to the video must be provided on the submission form on the Hackathon Website; and
 - must not include third party trademarks, or copyrighted music or other material unless the Entrant has permission to use such material.
- Track 1 & Track 2: Each submission must include a copy of the project's source code, either attached directly or linked to an open-source repository (e.g., GitHub).
- Track 3: Each submission must include proof artifacts (links/screenshots) as described in the track requirements.

Prizes

\$8,000 in prizes

Overall Winner

\$3,000 in cash

1 winner

Project featured in the Arm Community Blog.

Overall Runner Up

\$2,000 in cash

1 winner

Project featured in the Arm Community Blog.

Best in Category: Physical AI

\$1,000 in cash

1 winner

Project featured in the Arm Community Blog.

Best in Category: Cloud AI

\$1,000 in cash

1 winner

Project featured in the Arm Community Blog.

Best in Category: Mobile AI

\$1,000 in cash

1 winner

Project featured in the Arm Community Blog.

Judges

Avin Zarlez

Arm Staff SW Engineer - Developer Evangelist

Michael Hall

Arm Principal SW Engineer - Developer Evangelist

Gabriel Peterson

Arm Senior ML Engineer - Developer Evangelist

Rani Chowdary Mandepudi

Software Engineer, Strategy & Ecosystems, Arm

Judging Criteria

Technological Implementation – 40 points

Does the submission demonstrate quality software development? Does it clearly leverage Arm-powered platforms (on-device, Arm64, efficiency-minded design)? Is the technical approach sound and well executed?

User Experience / Developer Experience – 15 points

Is it clear how to use, run, or validate the project? Is the documentation well structured?

Could this be taken further or reused by other developers?

Potential Impact – 20 points

How useful is this to the developer community? Does it create reusable artifacts; optimized models, migration templates, prompt assets, or learning-ready content?

“WOW” factor – 25 points

How creative and compelling is the submission? Does it stand out in approach, usefulness, or clarity? Can it quickly capture attention and communicate value?

To help spark your creativity and provide valuable resources for your projects, we’ve compiled a list of inspirations and examples. These resources will guide you in building projects on Arm-powered platforms:

- [Arm Developer Program](https://developer.arm.com/): Join to access technical documentation, development tools, and community support. <https://developer.arm.com/>

- [Arm Learning paths](https://learn.arm.com/): Learn Arm architecture and AI topics through guided Learning paths.
- [Arm Developer Ecosystem GitHub](https://github.com/ArmDeveloperEcosystem): Open-source projects and code examples for inspiration and reuse.

During the challenge we will run workshops and office hours. Details will be shared in the [Arm Developer Program Discord](#), where you can reach Arm engineers and evangelists for support. Good luck, and we can't wait to see what you create!

Project Requirements

- **What to Create:** Entrants must create, migrate, or optimize an AI solution on Arm architecture and fall within at least one of these tracks (each a "Project").
- **Choose a Track:** Each submission must clearly select one of the three tracks (Track 1, Track 2, or Track 3).
 1. **Track 1 Optimization output**
Each submission must include a copy of the project's source code, either attached directly or linked to an open-source repository (e.g., GitHub).
 2. **Track 2 Migration/adoption value**
Each submission must include a copy of the project's source code, either attached directly or linked to an open-source repository (e.g., GitHub).
 3. **Track 3 Scale + learning completion for Source Code / Artifacts:**
Each submission must include proof artifacts (links/screenshots) as described in the track requirements.
- **Project Write-Up:** Along with source code or proof artifacts, a detailed write-up must be included. This write-up should cover:
 1. **Project Overview:** A brief description of the project and its purpose. Also explain what makes it interesting and why it should win.
 2. **Functionality / Output:** Explain what the project does and what the final output is (optimized model, migration example, scavenger deliverables, etc.).
 3. **Setup Instructions:** Step-by-step instructions on how to build/run/validate on an Arm-powered device or Arm64 environment (as applicable to your track).
- **Optional Media:** Participants are encouraged to include an audio or video recording that explains or pitches the project. A clear demo can significantly help judges understand your work.
- **Functionality:** The Project must be capable of being successfully installed and running consistently on the platform for which it is intended and must function as depicted in the video and/or expressed in the text description.
- **Platforms:** A submitted Project must run on the platform for which it is intended and which is specified in the Submission Requirements.
- **New & Existing:** Projects must be either newly created by the Entrant or, if the Entrant's Project existed prior to the Hackathon Submission Period, must have been significantly updated after the start of the Hackathon Submission Period. Entrants should explain how their Project was significantly updated during the Submission Period.

- Third Party Integrations: If a Project integrates any third-party SDK, APIs and/or data, Entrant must be authorized to use them in accordance with any terms and conditions or licensing requirements of the tool.

6. Judges & Criteria

Eligible submissions will be evaluated by a panel of judges selected by the Sponsor (the “Judges”). Judges may be employees of the sponsor or third parties, may or may not be listed individually on the Hackathon Website, and may change before or during the Judging Period. Judging may take place in one or more rounds with one or more panels of Judges, at the discretion of the sponsor.

Stage One) The first stage will determine via pass/fail whether the ideas meet a baseline level of viability, in that the Project reasonably fits the theme and reasonably applies the required APIs/SDKs featured in the Hackathon.

Stage Two) All Submissions that pass Stage One will be evaluated in Stage Two based on the following weighted criteria (the “Judging Criteria”):

Entries will be judged on the following equally weighted criteria, and according to the sole and absolute discretion of the judges:

- Technological Implementation – 40 points
Does the submission demonstrate quality software development? Does it clearly leverage Arm-powered platforms (on-device, Arm64, efficiency-minded design)? Is the technical approach sound and well executed?
- User Experience / Developer Experience – 15 points
Is it clear how to use, run, or validate the project? Is the documentation well structured? Could this be taken further or reused by other developers?
- Potential Impact – 20 points
How useful is this to the developer community? Does it create reusable artifacts; optimized models, migration templates, prompt assets, or learning-ready content?
- “WOW” factor – 25 points
How creative and compelling is the submission? Does it stand out in approach, usefulness, or clarity? Can it quickly capture attention and communicate value?

Track alignment note: Judges may also consider whether the submission strongly meets the spirit of its selected track (optimization output for Track 1, migration/adoption value for Track 2, scale + learning completion for Track 3).

The scores from the Judges will determine the potential winners of the applicable prizes. The Entrant(s) that are eligible for a Prize, and whose Submissions earn the highest overall scores based on the applicable Judging Criteria, will become potential winners of that Prize.

Tie Breaking

For each Prize listed below, if two or more Submissions are tied, the tied Submission with the highest score in the first applicable criterion listed above will be considered the higher scoring Submission. In the event any ties remain, this process will be repeated, as needed, by comparing the tied Submissions’ scores on the next applicable criterion. If two or more

Submissions are tied on all applicable criteria, the panel of Judges will vote on the tied Submissions.

[Arm Create: AI Optimization Challenge Getting Started Resources](#)

Hello everyone! Thank you for signing up for the Arm AI Optimization Challenge. My name is Avin Zarlez, and I'm a Staff Developer Evangelist at Arm. I'll be helping support participants throughout the challenge and wanted to share a few resources as you begin planning and building your project.

The [Arm Developer Program](#) is a great place to access Arm tools, learning resources, support, and community spaces. We also encourage you to join the Arm Developer Program Discord so you can discuss project ideas, share progress, ask technical questions, and connect with other participants and Arm engineers.

As you work on your submission, we encourage if you are using Neoverse/Cloud based Arm offerings to use [Arm Performix](#) to measure and validate the impact of your optimizations. Performix can help identify bottlenecks, capture performance data, and show measurable improvements. You can start with the [Arm Performix User Guide](#) and the [Arm Performix Learning Path](#).

For challenge-specific questions, you can post in the [Devpost Discussion section](#) or ask in Discord. If you have questions, feedback, or concerns that the Arm team can help address, please let us know. We're looking forward to seeing what you build and how you choose to optimize AI on Arm.

Track 1: Physical AI

Physical AI is AI that interacts with the physical world, such as robotics, autonomous vehicles, drones, industrial systems, and embedded intelligent devices. Unlike Cloud AI, Physical AI does not only generate outputs for a user or another software service. It must perceive real-world inputs from sensors and produce actions, control signals, or decisions that affect a machine operating in a physical environment.

Eligible projects may include one or more of the following:

- AI inference running on-device, on-robot, in-vehicle, or on nearby edge compute
- Applications that use real or simulated sensor data, such as camera, lidar, radar, IMU, GPS, audio, force, or other environmental signals
- Software that produces actions such as steering, braking, path planning, object manipulation, anomaly detection, navigation, alerting, or actuator control
- Systems combining AI with robotics or autonomy software stacks, such as ROS 2, simulation environments, middleware, or real-time control frameworks

This track includes AI workloads for:

- Robotics, such as robot perception, navigation, manipulation, motion control, voice interaction, and multi-robot coordination
- Automotive and autonomous systems, such as driver assistance, perception, planning, simulation, localization, safety monitoring, and in-vehicle AI pipelines

- Embedded and edge AI devices, such as smart cameras, sensor hubs, industrial controllers, drones, wearable devices, or intelligent appliances

Projects may run on Arm-based embedded, edge, or automotive platforms, including robotics SoCs, embedded Linux boards, microcontroller platforms, in-vehicle compute, or Arm-based edge servers used for local autonomy workloads.

Learning paths:

Robotics / middleware

- [ROS 2 install guide](#)
- [Cyclone DDS install guide](#)
- [Robot Simulation and Reinforcement Learning Workflows on Arm](#)
- [Deploy multi-node Zenoh systems on Raspberry Pi](#)

Automotive / autonomous driving

- [Deploy Open AD Kit containerized autonomous driving simulation on Arm Neoverse](#)
- [Prototype safety-critical isolation for autonomous driving systems on Arm Neoverse](#)

Embedded / Edge AI

- [Visualize Ethos-U NPU performance with ExecuTorch](#)

Track 2: Cloud AI

Cloud AI is any kind of inference run on cloud compute infrastructure, such as AWS, Azure, or GCP, or on a dedicated server on-prem. Cloud AI is typically accessed as an API endpoint by other software, or with a user-accessible frontend. Optimizing AI for the cloud means looking at scalability and efficiency on the most cost-effective compute instances, which offer higher core count and larger memory than Physical or Mobile AI.

Eligible projects may include one or more of the following:

- AI Inference running on Arm-based compute, such as AWS Graviton, Microsoft Cobalt, GCP Axion, or Ampere based servers
- Applications optimized for CPU-based inference through processes of quantization or pruning that allow running more complex models on standard CPU-only cloud instances
- Using CPU-optimized frameworks such as ExecuTorch, LiteRT, and Llama.cpp to accelerate AI inference workloads on CPU-only instances

This track includes AI workloads for:

- Agentic workloads that combine multiple AI models, MCP servers, and integrations to complete complex tasks with little to no human intervention
- AI features like chat, summarization, classification, or transformation that can be scaled-out using the same cloud-native tools and techniques as the rest of the application stack

Learning paths:

- [Deploy an AI Agent on Arm with Llama.cpp and Llama.cpp-agent using KleidiAI](#)
- [Deploy a Large Language Model \(LLM\) chatbot with Llama.cpp using KleidiAI on Arm servers](#)
- [Deploy a LLM-based Vision Chatbot with PyTorch and Hugging Face Transformers on Google Axion processors](#)
- [Accelerate Whisper on Arm with Hugging Face Transformers](#)
- [Run vLLM inference with INT4 quantization on Arm servers](#)

- [Run a Large Language Model \(LLM\) chatbot with PyTorch using KleidiAI on Arm servers](#)
- [Run an LLM chatbot with rtp-llm on Arm-based servers](#)
- [Deploy SqueezeNet 1.0 INT8 model with ONNX Runtime on Azure Cobalt 100](#)
- [Deploy TensorFlow on Google Cloud C4A \(Arm-based Axion VMs\)](#)
- [Build a Multimodal Retail Restocking Assistant on Armv9 With MNN](#)

Track 3: Mobile AI

Mobile AI is AI that runs locally on Arm-powered client devices such as smartphones, tablets, and laptops. Unlike Cloud AI, Mobile AI performs inference on the device in the user's hands, enabling low-latency, private, and offline-capable experiences for real applications.

Eligible projects may include one or more of the following:

- AI inference running fully on-device in a mobile or client application, such as text, vision, speech, or multimodal use cases
- Applications optimized for mobile constraints such as model size, memory use, responsiveness, battery awareness, offline use, or time to first token
- Projects using mobile or client inference frameworks such as ExecuTorch, ONNX Runtime, LiteRT / TensorFlow Lite, MediaPipe, or similar runtimes
- Cross-platform AI applications that run on Android, iOS, or Windows on Arm devices, as long as inference runs locally on Arm-powered hardware

This track includes AI workloads for:

- Mobile phones and tablets, such as chat, summarization, translation, transcription, camera intelligence, and accessibility features
- Laptops and PCs, such as Windows on Arm systems running local copilots, creative tools, productivity features, or developer assistants
- Consumer on-device AI experiences that benefit from local execution, privacy, responsiveness, and reduced cloud dependence

Learning paths:

On-device LLM applications (Android)

- [Build an Android chat app with Llama, KleidiAI, ExecuTorch, and XNNPACK](#)
- [Build a customer support chatbot on Android with Llama and ExecuTorch](#)

Mobile AI frameworks and runtimes

- [Run LLM inference on Android with KleidiAI, MediaPipe, and XNNPACK](#)

Performance and optimization on-device

- [Measure LLM inference performance with KleidiAI and SME2 on Android](#)